In [1]:
```
import random
import numpy as np
from data_process import get_CIFAR10_data
import math
from scipy.spatial import distance
from models import KNN, Perceptron, SVM, Softmax
from kaggle_submission import output_submission_csv
%matplotlib inline
```

Loading CIFAR-10

In the following cells we determine the number of images for each split and load the images.

In [2]:
```
# You can change these numbers for experimentation
# For submission we will use the default values
TRAIN_IMAGES = 49000
VAL_IMAGES = 1000
TEST_IMAGES = 5000
```

In [3]:
```
data = get_CIFAR10_data(TRAIN_IMAGES, VAL_IMAGES, TEST_IMAGES)
X_train, y_train = data['X_train'], data['y_train']
X_val, y_val = data['X_val'], data['y_val']
X_test, y_test = data['X_test'], data['y_test']
```

```
C:\Users\anish\Downloads\assignment1\assignment1\data_process.py:16: VisibleDeprecat
ionWarning: dtype(): align should be passed as Python or NumPy boolean but got `alig
n=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  return  pickle.load(f, encoding='latin1')
```

Convert the sets of images from dimensions of **(N, 3, 32, 32) -> (N, 3072)** where N is the number of images so that each **3x32x32** image is represented by a single vector.

In [4]:
```
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
```

Get Accuracy

This function computes how well your model performs using accuracy as a metric.

In [5]:
```
def get_acc(pred, y_test):
    return np.sum(y_test==pred)/len(y_test)*100
```

K-Nearest Neighbors

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and selecting the class that is most common among the k most similar training examples

In this exercise you will implement these steps using writing efficient, vectorized code. Your final implementation should not use for loops to loop over each of the test and train examples. Instead, you should calculate distances between vectorized forms of the datasets. You may refer to the `scipy.spatial.distance.cdist` function to do this efficiently.

The following code :

- Creates an instance of the KNN classifier class with k = 5
- The train function of the KNN class is trained on the training data
- We use the predict function for predicting testing data labels

Training KNN

In [6]:
```
knn = KNN(5)
knn.train(X_train, y_train)
```

Find best k on validation

The value of k is an important hyperparameter for the KNN classifier. We will choose the best k by examining the performance of classifiers trained with different k values on the validation set.

It's not necessary to try many different values of k for the purposes of this exercise. You may increase k by a magnitude of 2 each iteration up to around k=100 or something similar to get a sense of classifier performance for different k values.

Modify the code below to loop though different values of k, train a KNN classifier for each k, and output the validation accuracy for each of the classifiers. Be sure to note your best k below as well.

In [7]:
```
k = 5
for x in range(k, 60, 2):
    k = x
    knn = KNN(k)
    knn.train(X_train, y_train)
    pred_knn = knn.predict(X_val)
    print('The validation accuracy is given by : %f' % (get_acc(pred_knn, y_val)))
    print(k)
```

```
The validation accuracy is given by : 32.600000
5
The validation accuracy is given by : 32.300000
7
The validation accuracy is given by : 30.800000
9
The validation accuracy is given by : 31.600000
11
The validation accuracy is given by : 30.600000
13
The validation accuracy is given by : 30.800000
15
The validation accuracy is given by : 31.200000
17
The validation accuracy is given by : 30.400000
19
The validation accuracy is given by : 30.400000
21
The validation accuracy is given by : 30.100000
23
The validation accuracy is given by : 30.400000
25
The validation accuracy is given by : 29.800000
27
The validation accuracy is given by : 28.900000
29
The validation accuracy is given by : 28.700000
31
The validation accuracy is given by : 28.900000
33
The validation accuracy is given by : 29.800000
35
The validation accuracy is given by : 29.500000
37
The validation accuracy is given by : 29.600000
39
The validation accuracy is given by : 29.300000
41
The validation accuracy is given by : 28.900000
43
The validation accuracy is given by : 28.600000
45
The validation accuracy is given by : 28.300000
47
The validation accuracy is given by : 28.700000
49
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[7], line 6
      4 knn = KNN(k)
      5 knn.train(X_train, y_train)
----> 6 pred_knn = knn.predict(X_val)
      7 print('The validation accuracy is given by : %f' % (get_acc(pred_knn, y_va
l)))
      8 print(k)

File ~\Downloads\assignment1\assignment1\models\KNN.py:26, in KNN.predict(self, X_te
st)
     22 def predict(self, X_test):
     23     """
     24     Predict labels for X_test using k nearest neighbors.
     25     """
---> 26     dist_ = self.find_dist(X_test)  # shape (num_test, num_train)
     28     # Indices of k nearest neighbors for each test example
     29     knn_idx = np.argsort(dist_, axis=1)[:, :self.k]  # (num_test, k)

File ~\Downloads\assignment1\assignment1\models\KNN.py:19, in KNN.find_dist(self, X_
test)
     15 """
     16 Returns a (num_test, num_train) distance matrix
     17 """
     18 # cdist(A, B) returns distances between each row of A and each row of B
---> 19 dist_ = scipy.spatial.distance.cdist(X_test, self.X_train, metric=
  )
     20 return dist_

File ~\Downloads\assignment1\.venv\Lib\site-packages\scipy\spatial\distance.py:2926,
in cdist(XA, XB, metric, out, **kwargs)
   2924 if metric_info is not None:
   2925     cdist_fn = metric_info.cdist_func
-> 2926     return cdist_fn(XA, XB, out=out, **kwargs)
   2927 elif mstr.startswith("test_"):
   2928     metric_info = _TEST_METRICS.get(mstr, None)

KeyboardInterrupt:
```

Testing KNN

Finally, once you have found the best k according to your experiments on the validation set, retrain a classifier with the best k and test your classifier on the test set.

In [8]:
```
best_k = 5
knn = KNN(best_k)
knn.train(X_train, y_train)
```

In [9]:
```
pred_knn = knn.predict(X_test)
print('The testing accuracy is given by : %f' % (get_acc(pred_knn, y_test)))
```

```
The testing accuracy is given by : 33.840000
```

KNN Kaggle Submission

Once you are satisfied with your solution and test accuracy output a file to submit your test set predictions to the Kaggle for Assignment 1 KNN. Use the following code to do so:

In [10]:
```
output_submission_csv('knn_submission.csv', knn.predict(X_test))
```

Perceptron

Perceptron has 2 hyperparameters that you can experiment with:

- **Learning rate** - controls how much we change the current weights of the classifier during each update. We set it at a default value of 0.5, but you should experiment with different values. We recommend changing the learning rate by factors of 10 and observing how the performance of the classifier changes. You should also try adding a **decay** which slowly reduces the learning rate over each epoch.
- **Number of Epochs** - An epoch is a complete iterative pass over all of the data in the dataset. During an epoch we predict a label using the classifier and then update the weights of the classifier according the perceptron update rule for each sample in the training set. You should try different values for the number of training epochs and report your results.

You will implement the Perceptron classifier in the **models/Perceptron.py**

The following code:

- Creates an instance of the Perceptron classifier class
- The train function of the Perceptron class is trained on the training data
- We use the predict function to find the training accuracy as well as the testing accuracy

Train Perceptron

In [37]:
```
percept_ = Perceptron()
percept_.train(X_train, y_train)
```

In [38]:
```
pred_percept = percept_.predict(X_train)
print('The training accuracy is given by : %f' % (get_acc(pred_percept, y_train)))
```

```
The training accuracy is given by : 36.165306
```

Validation

In [39]:
```
pred_percept = percept_.predict(X_val)
print('The validation accuracy is given by : %f' % (get_acc(pred_percept, y_val)))
```

```
The validation accuracy is given by : 30.000000
```

Test Perceptron

In [40]:
```
pred_percept = percept_.predict(X_test)
print('The testing accuracy is given by : %f' % (get_acc(pred_percept, y_test)))
```

```
The testing accuracy is given by : 30.700000
```

Perceptron Kaggle Submission

Once you are satisfied with your solution and test accuracy output a file to submit your test set predictions to the Kaggle for Assignment 1 Perceptron. Use the following code to do so:

In [60]:
```
output_submission_csv('perceptron_submission.csv', percept_.predict(X_test))
```

Support Vector Machines (with SGD)

Next, you will implement a "soft margin" SVM. In this formulation you will maximize the margin between positive and negative training examples and penalize margin violations using a hinge loss.

We will optimize the SVM loss using SGD. This means you must compute the loss function with respect to model weights. You will use this gradient to update the model weights.

SVM optimized with SGD has 3 hyperparameters that you can experiment with :

- **Learning rate** - similar to as defined above in Perceptron, this parameter scales by how much the weights are changed according to the calculated gradient update.
- **Epochs** - similar to as defined above in Perceptron.
- **Regularization constant** - Hyperparameter to determine the strength of regularization. In this case it is a coefficient on the term which maximizes the margin.

You will implement the SVM using SGD in the **models/SVM.py**

The following code:

- Creates an instance of the SVM classifier class
- The train function of the SVM class is trained on the training data
- We use the predict function to find the training accuracy as well as the testing accuracy

Train SVM

In [62]:
```
svm = SVM()
svm.train(X_train, y_train)
```

In [63]:
```
pred_svm = svm.predict(X_train)
print('The training accuracy is given by : %f' % (get_acc(pred_svm, y_train)))
```

```
The training accuracy is given by : 30.402041
```

Validate SVM

In [64]:
```
pred_svm = svm.predict(X_val)
print('The validation accuracy is given by : %f' % (get_acc(pred_svm, y_val)))
```

```
The validation accuracy is given by : 30.400000
```

Test SVM

In [65]:
```
pred_svm = svm.predict(X_test)
print('The testing accuracy is given by : %f' % (get_acc(pred_svm, y_test)))
```

```
The testing accuracy is given by : 29.380000
```

SVM Kaggle Submission

Once you are satisfied with your solution and test accuracy output a file to submit your test set predictions to the Kaggle for Assignment 1 SVM. Use the following code to do so:

In [67]:
```
output_submission_csv('svm_submission.csv', svm.predict(X_test))
```

Softmax Classifier (with SGD)

Next, you will train a Softmax classifier. This classifier consists of a linear function of the input data followed by a softmax function which outputs a vector of dimension C (number of classes) for each data point. Each entry of the softmax output vector corresponds to a confidence in one of the C classes, and like a probability distribution, the entries of the output vector sum to 1. We use a cross-entropy loss on this sotmax output to train the model.

Check the following link as an additional resource on softmax classification: http://cs231n.github.io/linear-classify/#softmax

Once again we will train the classifier with SGD. This means you need to compute the gradients of the softmax cross-entropy loss function according to the weights and update the weights using this gradient. Check the following link to help with implementing the gradient updates: https://deepnotes.io/softmax-crossentropy

The softmax classifier has 3 hyperparameters that you can experiment with :

- **Learning rate** - As above, this controls how much the model weights are updated with respect to their gradient.

- **Number of Epochs** - As described for perceptron.
- **Regularization constant** - Hyperparameter to determine the strength of regularization. In this case, we minimize the L2 norm of the model weights as regularization, so the regularization constant is a coefficient on the L2 norm in the combined cross-entropy and regularization objective.

You will implement a softmax classifier using SGD in the **models/Softmax.py**

The following code:

- Creates an instance of the Softmax classifier class
- The train function of the Softmax class is trained on the training data
- We use the predict function to find the training accuracy as well as the testing accuracy

Train Softmax

In [49]:
```
softmax = Softmax()
softmax.train(X_train, y_train)
```

In [50]:
```
pred_softmax = softmax.predict(X_train)
print('The training accuracy is given by : %f' % (get_acc(pred_softmax, y_train)))
```

```
The training accuracy is given by : 32.469388
```

Validate Softmax

In [51]:
```
pred_softmax = softmax.predict(X_val)
print('The validation accuracy is given by : %f' % (get_acc(pred_softmax, y_val)))
```

```
The validation accuracy is given by : 29.100000
```

Testing Softmax

In [52]:
```
pred_softmax = softmax.predict(X_test)
print('The testing accuracy is given by : %f' % (get_acc(pred_softmax, y_test)))
```

```
The testing accuracy is given by : 30.440000
```

Softmax Kaggle Submission

Once you are satisfied with your solution and test accuracy output a file to submit your test set predictions to the Kaggle for Assignment 1 Softmax. Use the following code to do so:

In [58]:
```
output_submission_csv('softmax_submission.csv', softmax.predict(X_test))
```